USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Build and Flash Pipeline

UM-NATOS-005

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-005	1.0 · 2026-08-14	current	Internal

1. Abstract

This report documents how nat-os source becomes a bootable image, every compiler and linker flag and why it is present, why PlatformIO is used as a toolchain source but not as a build system, and how to reproduce a build and capture boot output.

2. What PlatformIO is and is not

PlatformIO is a **build orchestrator and package manager**. It downloads toolchains, resolves libraries, invokes the compiler and linker, and calls `esptool`. Nothing it produces is required on the target.

nat-os continues to use PlatformIO as a **source of tools**:

Tool	Path
`xtensa-esp32-elf-gcc` (14.2.0)	`~/.platformio/packages/toolchain-xtensa-esp32/bin/`
`xtensa-esp32-elf-objcopy`, `-objdump`, `-size`	same
`esptool.py` (4.5.1)	`~/.platformio/packages/tool-esptoolpy/`
Python with `pyserial`	`~/.platformio/penv/Scripts/python.exe`

It is **not** used to build, for one reason: PlatformIO builds are organised around a `framework` (`arduino` or `espidf`), and each framework links a substantial runtime — FreeRTOS, a heap, a C library, and a startup path that ends by calling user code. nat-os replaces all of that. The framework is therefore the obstacle rather than the foundation, and `-nostdlib -nostartfiles` says precisely that.

A framework-less PlatformIO configuration is possible but works against the tool's assumptions. For kernel development, every flag should be visible rather than inferred, which a 90-line script achieves and a `platformio.ini` does not.

3. Pipeline stages

```
kernel/*.c, *.S
      │  xtensa-esp32-elf-gcc  -c          (per translation unit)
      ▼
build/*.o
      │  xtensa-esp32-elf-gcc  -T linker.ld
      ▼
build/natos.elf            ← real addresses assigned here
      │  esptool elf2image
      ▼
build/natos.bin            ← bootloader-compatible image
      │  esptool write_flash 0x10000
      ▼
target board
```

4. Compiler flags

Flag	Purpose
`-mabi=call0`	Select the non-windowed ABI. **The load-bearing flag** — see UM-NATOS-003
`-mtext-section-literals`	Place literal pools inside `.text`. Without this they land in a separate section that the linker script does not map into IRAM, and `l32r` loads fault
`-mlongcalls`	Permit calls beyond the short-displacement range; required once code spans more than a small region
`-ffreestanding`	Tell GCC there is no hosted C environment — no `main` convention, no library assumptions
`-fno-builtin`	Prevent GCC from replacing loops with calls to `memcpy` / `memset`, which do not exist in this link
`-fno-stack-protector`	The stack guard requires runtime support that does not exist
`-Os`	Optimise for size; IRAM is the scarce resource
`-Wall -Wextra`	Warnings on. In a kernel, an unused-variable warning is often a real bug
`-std=c11`	Explicit language version

4.1 On `-mtext-section-literals` and the entry point

Xtensa loads large constants through a literal pool referenced by `l32r`. With this flag the pool is emitted at the head of `.text`. That is why the image entry point is `0x4008000C` rather than `0x40080000` — the first 12 bytes are literals, and `_start`'s first instruction follows them.

This also explains a disassembly artifact: `objdump` cannot distinguish literals from instructions, so disassembling from `0x40080000` shows plausible but meaningless instructions before the real code. Worth remembering when reading a fault address.

5. Linker flags

Flag	Purpose
`-nostdlib`	Link no standard library
`-nostartfiles`	Link no C runtime startup — `_start` is ours
`-T kernel/linker.ld`	Our memory map (UM-NATOS-004)
`-Wl,--gc-sections`	Discard unreferenced sections
`-Wl,-Map,build/natos.map`	Emit a map file — the authoritative record of what landed where
`-mabi=call0`	Required on the link line too, so the driver selects compatible internals

6. Packaging and flashing

```
esptool --chip esp32 elf2image --flash_mode dio --flash_freq 40m --flash_size 4MB -o natos.bin natos.elf
```

Flash write targets three regions:

Offset	Contents
`0x1000`	Borrowed second-stage bootloader
`0x8000`	Borrowed partition table
`0x10000`	`natos.bin`

The two borrowed binaries are taken from the CYD PlatformIO project's build directory. They change only if that project is rebuilt, and could be copied into this repository to remove the external dependency — **recommended**, since the current arrangement means deleting an unrelated project's build directory breaks this one's flash step.

7. Reproduction

```
cd C:\Users\nobod\Projects\nat-os
.\build.ps1                                 # build only
.\build.ps1 -Flash -Port COM5               # build and flash
.\build.ps1 -Flash -Monitor -Port COM5      # build, flash, attach monitor
```

8. Capturing boot output

Attaching a monitor *after* reset loses the banner, because the kernel prints within milliseconds of the jump. The capture harness therefore opens the port first and then pulses the auto-reset circuit:

- `RTS` → `EN` (reset), asserted then released
- `DTR` → `GPIO0` (boot mode), held high for normal flash boot

Sequence: open port → `DTR=False`, `RTS=True` → 150 ms → `RTS=False` → read.

This guarantees the ROM trace and reset reason are captured, which §7 of UM-NATOS-002 identifies as the highest-value diagnostic when a boot fails.

9. Defects encountered and resolved

Recorded because both are easy to hit again.

Defect	Symptom	Resolution
Unquoted `-Wl,` arguments	PowerShell parse error before GCC ran; "Missing argument in parameter list"	Quote every `-Wl,...` string — PowerShell reads the comma as an array separator
Wrong Python interpreter	`ModuleNotFoundError: No module named 'serial'` during **offline** `elf2image`	Use PlatformIO's `penv` interpreter. `esptool` imports `serial` unconditionally, even for operations that touch no serial port

10. Known gaps

- **No dependency tracking.** Every build recompiles every file. Trivial at three files; will need addressing.
- **No incremental or parallel build.**
- **No test target.** There is no host-side unit test path; everything is verified on hardware.
- **Hard-coded paths.** `build.ps1` contains absolute paths to the borrowed binaries and the toolchain root.

11. References

- UM-NATOS-003 — why `-mabi=call0` is mandatory and cannot be mixed
- UM-NATOS-004 — the memory map the linker script implements
- UM-NATOS-002 §8 — reproduction in the context of the boot chain